

Week 3 Lab — Control Flow & Functions

Name: HAKIZIMANA Honore Patient

Student ID: 29134

Program: Software Engineering

Part 1 · Making Decisions (5 marks)

Exercise 1.1 — The grading ladder (3 marks)

```
score = 91

if score >= 85:
    print("A")
elif score >= 70:
    print("B")
elif score >= 60:
    print("C")
elif score >= 50:
    print("D")
else:
    print("F")

# Test results:
# 91 → A
# 76 → B
# 64 → C
# 52 → D
# 43 → F
```

Exercise 1.2 — Combined conditions (2 marks)

```
score = 76
attendance = 92

if score >= 50 and attendance >= 75:
    print("Validated")
elif score >= 50 and attendance < 75:
    print("Blocked: low attendance")
else:
    print("Failed")

# Test results:
# (76, 92) → Validated
# (76, 68) → Blocked: low attendance
# (43, 95) → Failed
```

Question 1.3 — Think about it (included in Part 1 marks)

Answer

If `elif score >= 50:` came before `elif score >= 70:`, scores from 70 upward would be classified as D before the program could check for B.

Part 2 · Loops & the Accumulator (7 marks)

Exercise 2.1 — Loop + filter (2 marks)

```
scores = [72, 85, 91, 64, 78, 47, 88, 55, 93, 61]

for score in scores:
    if score >= 80:
        print("Top score:", score)

# Output:
# Top score: 85
# Top score: 91
# Top score: 88
# Top score: 93
```

Exercise 2.2 — The accumulator pattern (3 marks)

```
scores = [72, 85, 91, 64, 78, 47, 88, 55, 93, 61]

total = 0
count = 0
passed = 0
failed = 0

for score in scores:
    total = total + score
    count = count + 1

    if score >= 50:
        passed = passed + 1
    else:
        failed = failed + 1

average = total / count

print("Total:", total)
print("Count:", count)
print("Average:", average)
print("Passed:", passed)
print("Failed:", failed)

# Output:
# Total: 734
# Count: 10
# Average: 73.4
# Passed: 9
# Failed: 1
```

Exercise 2.3 — while loop (2 marks)

```
saved = 0
months = 0
target = 250000

while saved < target:
    saved = saved + 15000
    months = months + 1

print("Months needed:", months)
print("Amount saved:", saved)

# Output:
# Months needed: 17
# Amount saved: 255000
```

Part 3 · Functions (6 marks)

Exercise 3.1 — get_grade (3 marks)

```
def get_grade(score):
    """Return the letter grade for a score."""

    if score >= 85:
        return "A"
    elif score >= 70:
        return "B"
    elif score >= 60:
        return "C"
    elif score >= 50:
        return "D"
    else:
        return "F"

print(get_grade(91))
print(get_grade(76))
print(get_grade(43))

# Output:
# A
```

```
# B
# F
```

Exercise 3.2 — pass_rate (3 marks)

```
def pass_rate(score_list):
    passed = 0
    count = 0

    for score in score_list:
        count = count + 1
        if score >= 50:
            passed = passed + 1

    return (passed / count) * 100

scores = [72, 85, 91, 64, 78, 47, 88, 55, 93, 61]
print(pass_rate(scores))

# Output:
# 90.0
```

Part 4 · Reading the File (5 marks)

Exercise 4.1 — First contact (2 marks)

```
with open("week3_students.csv", "r") as f:
    lines = f.readlines()

print("Number of lines:", len(lines))
print("Header:", lines[0].strip())
print("First student's line:", lines[1].strip())

# There are 41 lines because there is 1 header line
# plus 40 student records.
```

Exercise 4.2 — Extract a column (3 marks)

```
with open("week3_students.csv", "r") as f:
    lines = f.readlines()

names = []

for line in lines[1:]:
    parts = line.strip().split(",")
    names.append(parts[1])

print("Count:", len(names))
print("First 5 names:", names[:5])

# Count: 40
# First 5:
# Aline Uwase
# Eric Niyonzima
# Diane Mukamana
# Jean Bosco Habimana
# Sandrine Ingabire
```

Part 5 · The Resilient Pipeline (12 marks)

Exercise 5.1 — Survive the mess (4 marks)

```
with open("week3_students.csv", "r") as f:
    lines = f.readlines()

total = 0
valid = 0
bad = 0
bad_ids = []

for line in lines[1:]:
    parts = line.strip().split(",")

    try:
```

```

        score = int(parts[7])
        total = total + score
        valid = valid + 1
    except ValueError:
        bad = bad + 1
        bad_ids.append(parts[0])

average = total / valid

print("Valid:", valid)
print("Bad:", bad)
print("Bad IDs:", bad_ids)
print("Average:", round(average, 1))

# Expected:
# Valid: 37
# Bad: 3
# Bad IDs: the 3 IDs found in the CSV

```

Exercise 5.2 — Grades & the top student (4 marks)

```

with open("week3_students.csv", "r") as f:
    lines = f.readlines()

passed = 0
failed = 0
best_score = -1
best_name = ""

for line in lines[1:]:
    parts = line.strip().split(",")

    try:
        score = int(parts[7])

        if score >= 50:
            passed = passed + 1
        else:
            failed = failed + 1

        if score > best_score:
            best_score = score
            best_name = parts[1]

    except ValueError:
        continue

print("Passed:", passed)
print("Failed:", failed)
print("Best score:", best_score)
print("Top student:", best_name)
print("Grade:", get_grade(best_score))

```

Exercise 5.3 — Stretch: report by district (4 marks)

```

with open("week3_students.csv", "r") as f:
    lines = f.readlines()

sums = {}
counts = {}

for line in lines[1:]:
    parts = line.strip().split(",")

    try:
        s = int(parts[7])
        d = parts[5]

        sums[d] = sums.get(d, 0) + s
        counts[d] = counts.get(d, 0) + 1

    except ValueError:
        continue

for district in sums:

```

```
average = sums[district] / counts[district]
print(district, ":", round(average, 1))
```

Bonus (+3) — Your first output file

```
with open("week3_report.txt", "w") as f:
    f.write("Week 3 Big Data Lab Report\n")
    f.write("Student: HAKIZIMANA Honore Patient\n")
    f.write("Student ID: 29134\n")
    f.write("Program: Software Engineering\n")
    f.write("Student count: 40\n")
    f.write("Valid scores: 37\n")
    f.write("Bad scores: 3\n")
    f.write("Average: " + str(round(average, 1)) + "\n")
    f.write("Top student: " + best_name + "\n")

with open("week3_report.txt", "r") as f:
    report = f.read()

print(report)
```

Part 6 · Reflection (required, checked for completion)

1. Your pipeline skipped 3 broken records out of 40 (7.5%). At what percentage would you stop trusting the dataset — and what would you do instead of skipping?

I would stop trusting the dataset if more than about 20% of the records were invalid. At that point, I would investigate the source of the errors, correct or recover the missing values, and request a clean dataset instead of simply skipping the records.

2. Describe one thing from today's session that surprised or confused you.

One thing that surprised me was how a single invalid value in a CSV file can stop a program when converting data to an integer. I learned that using try and except ValueError allows the program to handle the bad record and continue processing the valid records.

Submission Checklist

Checklist

- Notebook renamed Week3_YourName
- All exercises run without errors
- 3 broken records identified
- Reflection answered
- Screenshots
- Short report
- Submitted before end of Week 3